

思考题，内核用 esp 定位现运行进程的 user 结构，改写内核函数 GetUser ()

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博

学号 2351520

当前实现：

- GetUser() 返回固定虚拟地址 0xC03FF000 处的 User 结构
- 进程切换时，SwitchUStruct(p) 宏修改页表第 1023 项，将该虚拟地址映射到新进程的 PPDA 物理页

```
User& Kernel::GetUser()
{
    return *(User*)USER_ADDRESS; // USER_ADDRESS = 0xC03FF000
}
```

Linux 的 ESP 定位方法 (Linus 的做法)：

- 每个进程的 User 结构位于内核栈页的底部
- 内核栈向下增长，ESP 总是指向该页内
- 用 ESP 与 $\sim(\text{PAGE_SIZE} - 1)$ 进行与运算，即可得到 User 结构的地址
-

UnixV6++ 的内存布局：

- PPDA 大小为 4KB (USIZE = 0x1000)
- User 结构位于 PPDA 开始处
- 内核栈从 PPDA 顶部向下增长

因此，无论 ESP 指向页内哪个位置，清除低 12 位后，都能得到页的起始地址（即 User 结构地址）： $\text{ESP} \& \sim(0x1000 - 1) = \text{ESP} \& 0xFFFFF000$ 就能得到 User 结构的虚拟地址。

根据这个思路，改写 GetUser()，并且在 GetUser 中输出调试信息来验证正确性：

```
User& Kernel::GetUser()
{
    unsigned long esp;
    __asm__ __volatile__ ("movl %%esp, %0" : "=r"(esp));

    User* pUser = (User*)(esp & ~((ProcessManager::USIZE - 1)));

    /* 验证：ESP 方法与固定地址方法结果应一致 */
    if ((unsigned long)pUser != USER_ADDRESS){
        Diagnose::Write("GetUser ERROR: ESP=0x%x, pUser=0x%x, expected=0x%x\n",
            esp, (unsigned long)pUser, USER_ADDRESS);
    }
    else {
```

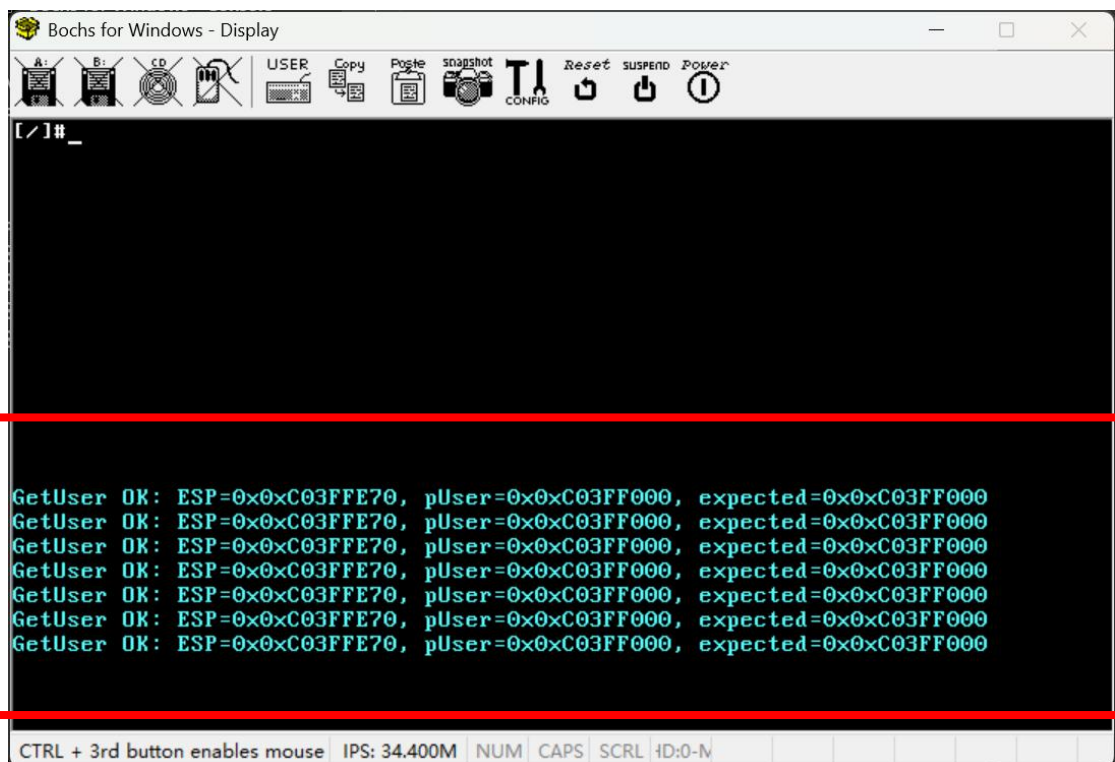
```

        Diagnose::Write("GetUser OK: ESP=0x%x, pUser=0x%x, expected=0x%x\n",
            esp, (unsigned long)pUser, USER_ADDRESS);
    }

    return *pUser;
}

```

重新编译运行系统后，调试窗口被刷屏，不断打印 "GetUser OK"。这证明了修改后的用 `esp` 定位现运行进程的 `user` 结构的方法和原先写死定位的结果是一致的。至于为什么会被刷屏，因为 `Kernel::GetUser()` 是内核中最核心的函数之一，用于获取当前进程的上下文。系统运行时的每一次系统调用、中断处理、进程切换等都会调用它。所以 `GetUser()` 调用频率极高。



```

Bochs for Windows - Display
A: B: CD USER Copy Paste snapshot CONFIG Reset SUSPEND Power
[ / ] # _

GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000
GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000
GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000
GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000
GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000
GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000
GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000
GetUser OK: ESP=0x0xC03FFE70, pUser=0x0xC03FF000, expected=0x0xC03FF000

CTRL + 3rd button enables mouse  IPS: 34.400M  NUM  CAPS  SCRL  ID:0-N

```